

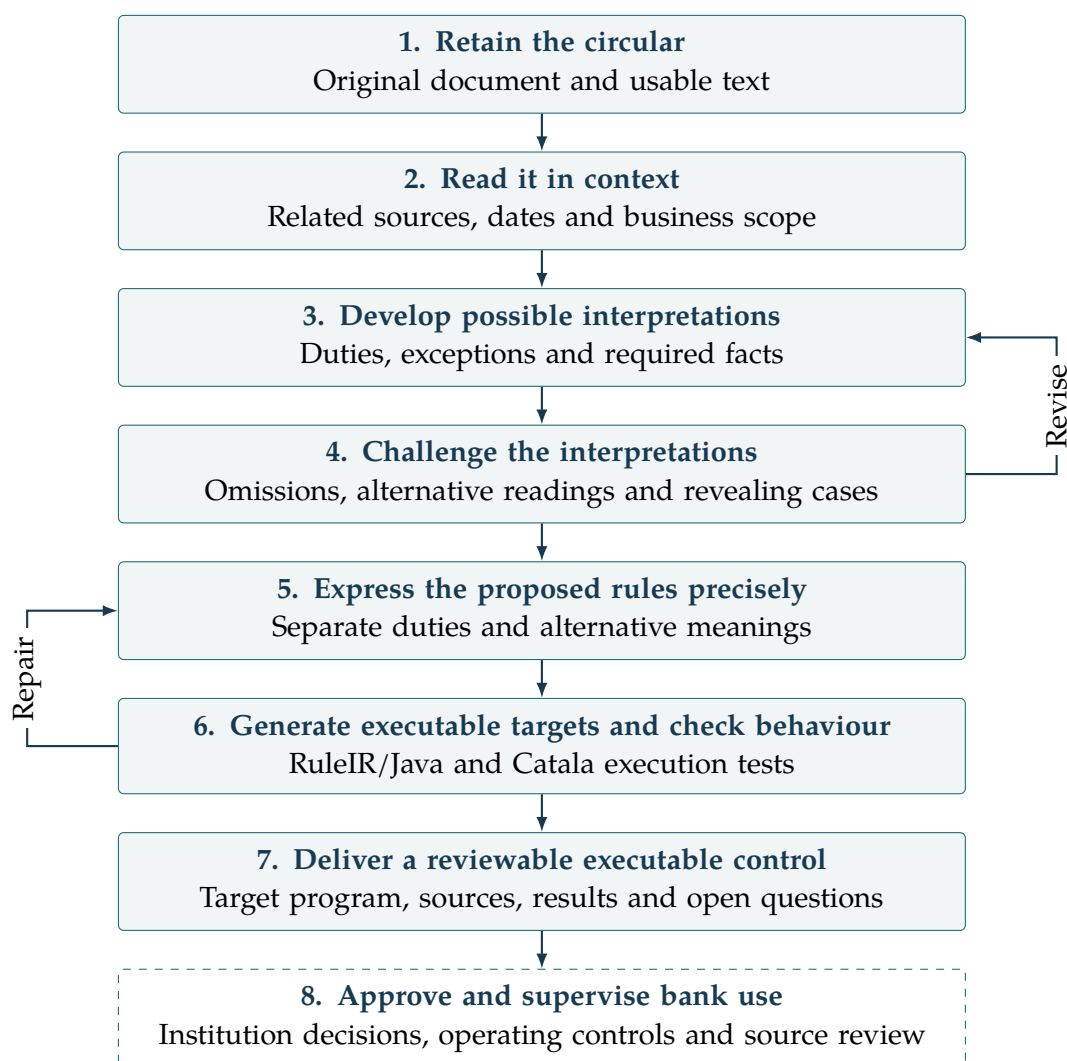
From the circular to executable controls

A circular supplies words; a Java program needs precise decisions. Between them lie questions about meaning, evidence and execution. The eight steps below locate that work. Each numbered box links to a closer view on the next pages, where the technology is introduced through the job it performs.

The first software stage, called the *frontend*, proposes a meaning for the passage and records its sources, assumptions and required facts. A *translator* then turns that fixed proposal into an executable language. LegalMath has two such targets. *RuleIR*, its own small rule language, expresses conditions such as a financial threshold and the effect of missing evidence. *Catala* is a programming language designed for legal rules, with a compiler that translates those rules into executable code. Both targets can receive the same proposed meaning; Catala also supports the implemented calculations over structured collections described in Chapter 8.

Stipula answers a different question: how does a contract change when a party takes an action? It describes parties, resources and states. For example, a request available while an agreement is active may become unavailable after withdrawal. The book studies Stipula's verification methods as a comparison; LegalMath does not currently compile Stipula contracts.

Eight steps from source to bank use

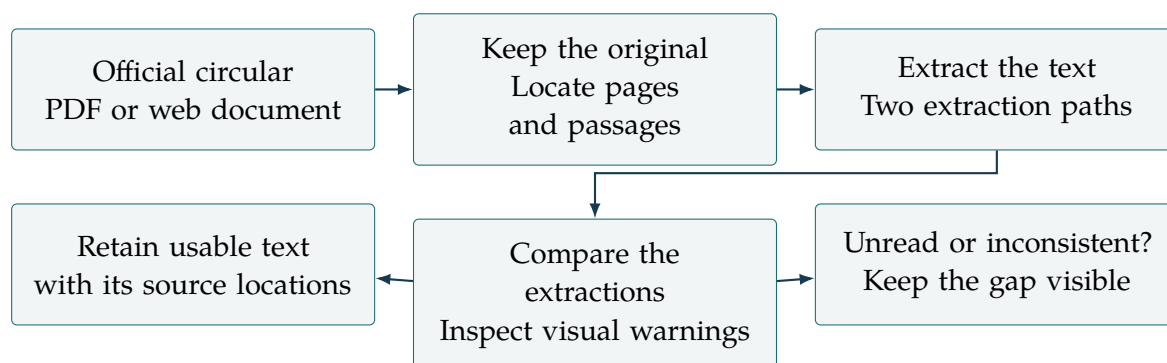


A tested program is a reviewable result. The dashed final step requires the bank's own authority and operating evidence.

The implemented route can produce draft controls while interpretations remain unresolved. It can also stop before code generation when a proposal cannot be expressed or evidence is missing. In the running example, whether a rebate is a fee discount can remain open even after its proposed rule runs correctly. The later charts show how that question stays attached to the result.

1. Retain the circular

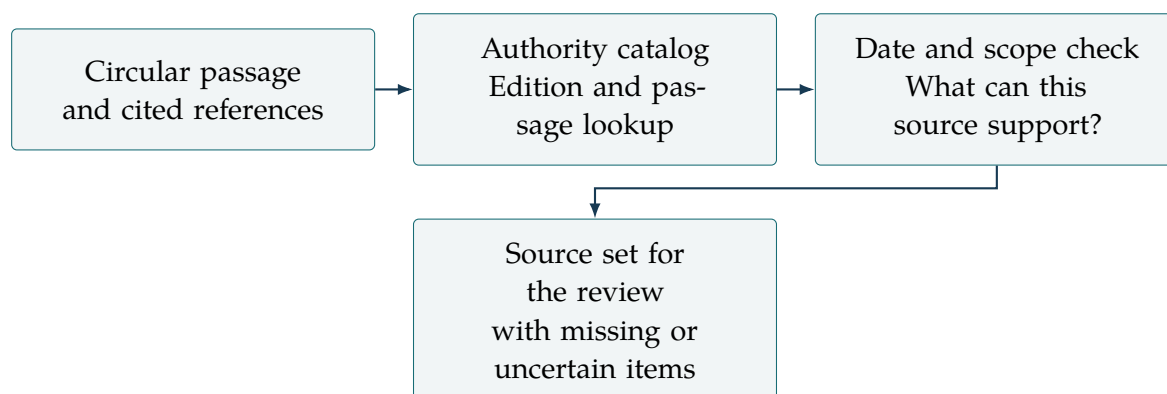
The first task is to give every later reading the same inspectable source. For PDFs, pypdf and Poppler's pdftotext are two different text extractors. Web pages have two parsing paths. Comparing their text can expose a missing line; checking the page for images can expose material that neither extractor has read.



Agreement between text extractors cannot reveal an exception present only in an image. The additional source checks in Chapter 6 read page images and compare the recognised text with the embedded text.

2. Read it in context

A gift restriction may refer to the Code of Conduct and a frequently asked question (FAQ). A retained authority catalog connects those references to document editions and passages. A dependency search follows explicit official links; missing documents remain named questions. The relevant business scope identifies the people, products, activities and dates being assessed.

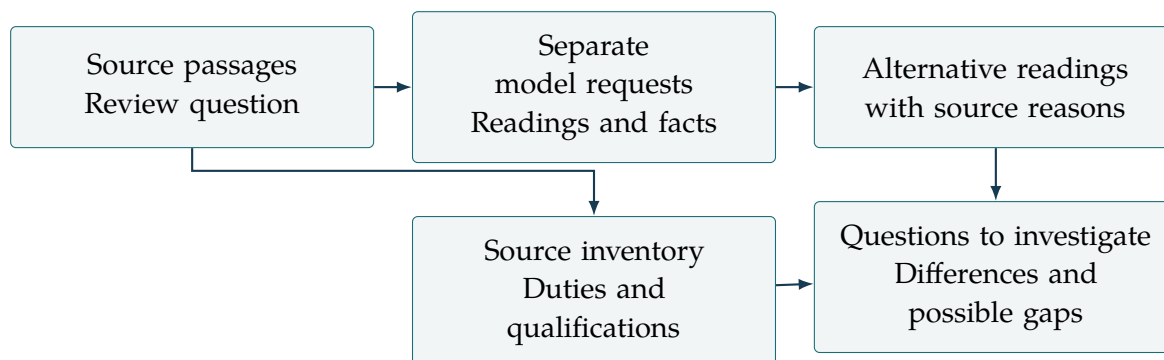


Finding the FAQ changes the question from where the source is to what its dated answer supports.

The retained FAQ page has later answer updates. Its older page date therefore does not establish everything a historical reader could have seen. A catalog match records useful evidence while preserving that uncertainty. Chapter 2 follows the actual Code and FAQ example; the companion's Appendix A gives the source tools.

3. Develop possible interpretations

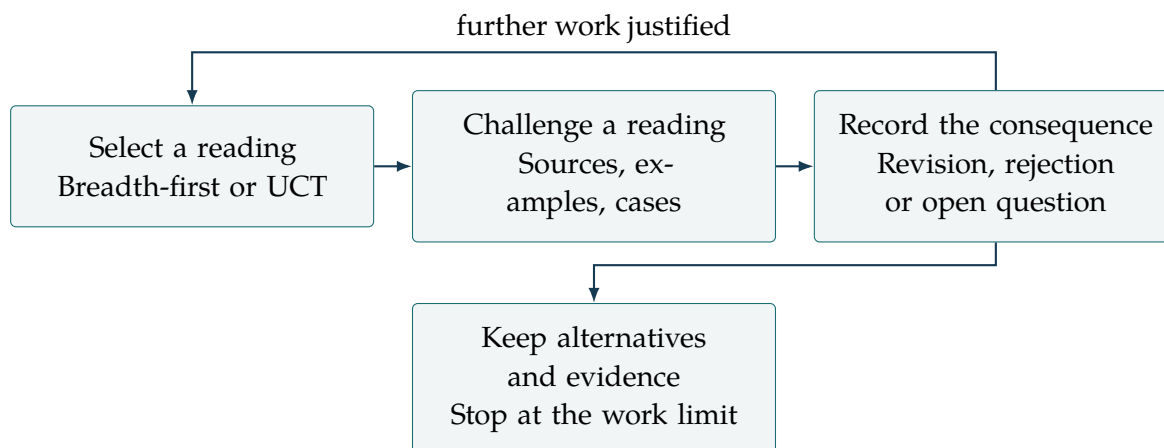
A language model can propose what a passage requires and which facts would decide a case. Separate requests produce initial readings without showing each request the other answers. Another set of requests inventories the source's requirements and qualifications. This second view helps expose an exception that all the proposed readings might miss.



Several readings reveal disagreement. A separate source inventory can reveal a shared omission. Separate requests can still share mistakes.

4. Challenge the interpretations

Tree search organises successive revisions of a reading. Breadth-first search visits the shallower alternatives first. Upper Confidence bounds applied to Trees (UCT) balances previously useful branches against less-visited ones. These are ways to choose the next investigation, not probabilities of legal correctness. Both are implemented with finite work limits.

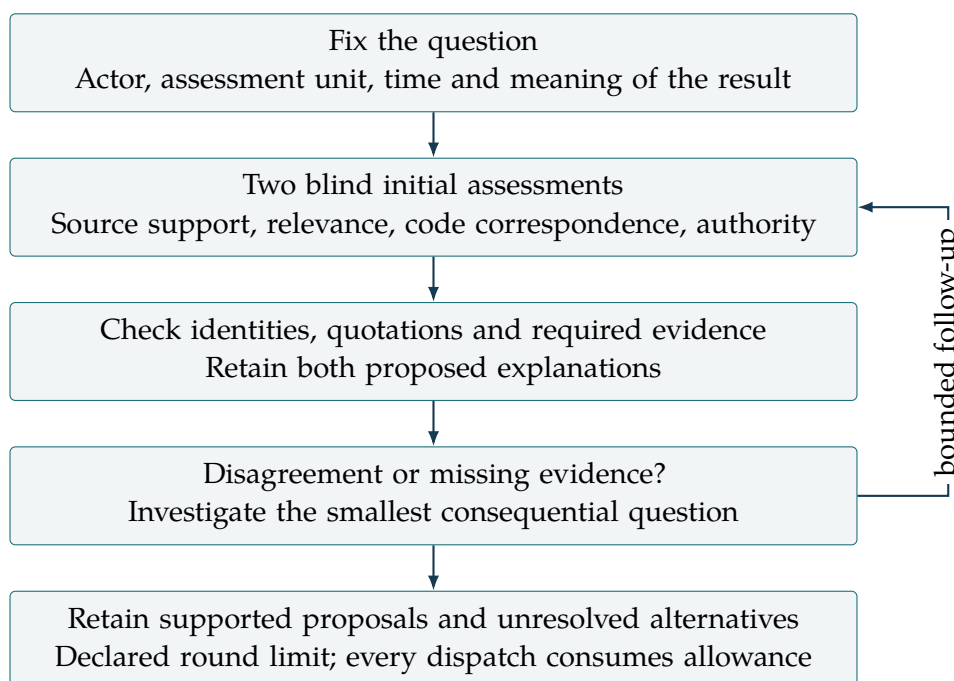


Search changes a proposed reading only with an explicit reason. Exhausting the work allowance leaves an unresolved result.

Typed arguments record support and objections; official examples retain their stated meaning. A rebate-and-voucher case can reveal where readings attach an exception. After aligning facts and outputs, the original programs execute that case. Draft compilation and step 6 therefore occur inside this loop too. Agreement still cannot establish complete source coverage. Chapters 5 and 6 develop these checks.

4 continued. Investigate the exact disagreement

The continuing investigation separates source support, relevance to the selected question, correspondence with the executable rule and availability of required authority. Two initial contexts work without peer answers. A later round can then address the precise disagreement, or repair an invalid response, within the same fixed allowance.



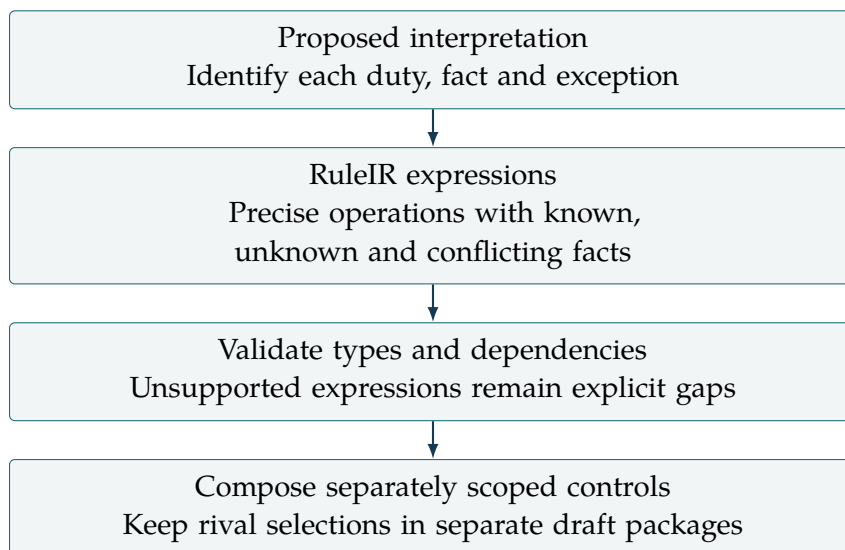
A repaired response is another proposal. Agreement cannot supply a missing source or prove an interpretation.

Rival factual models retain different actors, objects and assessment times. A proposed mapping permits a conditional comparison only after those distinctions and its assumptions are stated. Changed source bytes or methods start a new revision; the old program remains evidence for its old source. Section 6.35 follows the implemented distinctions through an STR contact history and the unfamiliar-circular study.

Capacity is checked separately from meaning. Smaller source or comparison batches retain the complete set of obligations and the original retry counts. The optional incremental check admits each validated part only while total retained size remains within its limit. A failed repair preserves earlier claims; it cannot replace them with an empty inventory. If the work still cannot finish, its missing checks remain in the final report.

5. Express the proposed rules precisely

A program needs the meaning of each fact and operation fixed. LegalMath's rule intermediate representation, **RuleIR**, is its small executable rule language. It records typed facts, conditions, exceptions and scope. Type checks reject such mistakes as adding a date to money; dependency checks reject missing or circular rule references.

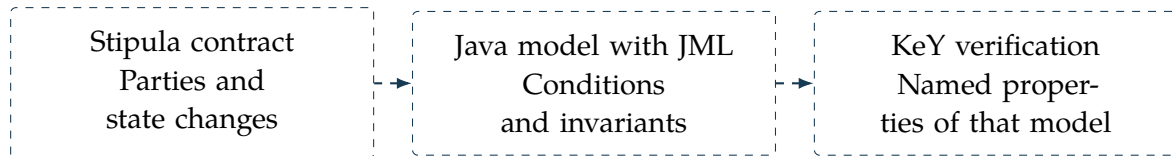


The current Java route starts from validated RuleIR. Successful construction does not decide which legal interpretation the bank should adopt.

For example, a gift restriction and a separately specified disclosure duty can both belong in a package. Two rival readings of the restriction remain alternative packages. The proposed grouping and each unresolved question are preserved. The common model now has two Catala routes. The compatibility route lowers a supported RuleIR profile and retains the existing result, trace and release checks. The native route consumes the shared model directly, so rich records, lists, options, payload variants and typed operations can be reported as executable Catala even when the RuleIR target returns an explicit capability failure. These routes share the frontend and evidence policy; they do not make different interpretations agree by construction.

5 continued. Where Stipula belongs

Stipula describes contracts through parties, resources, actions and state changes. It is relevant when consent can be granted and withdrawn. The published translation studied in Chapter 4 constructs a Java verification model with specifications in the Java Modeling Language (JML). KeY is a verifier that reasons about the Java model and those specifications.

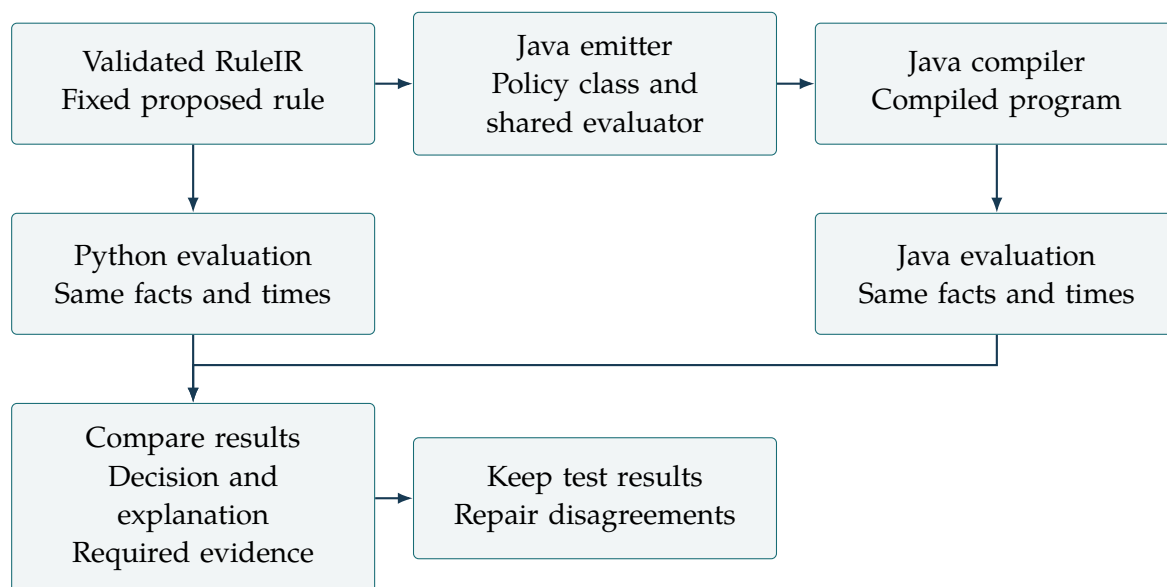


Dashed boxes show a studied research route, not the current LegalMath compiler. Its Java model is not automatically a bank application.

The retained translator examples contain a compilation discrepancy; no local KeY replay establishes their reported proofs. Adoption requires reproducible inputs and a justified mapping to LegalMath's event rules. Chapter 4 details those limits; Chapter 7 explains the implemented rule language.

6. Generate executable targets and check their behaviour

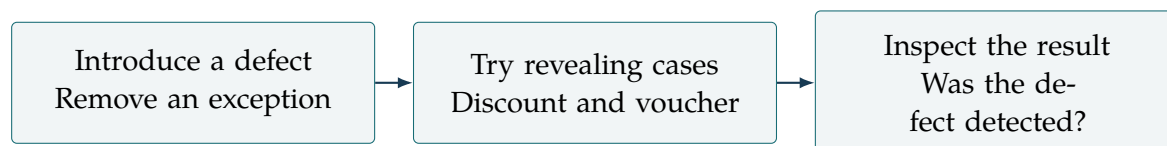
The Java emitter embeds the fixed RuleIR rules in a generated policy class that calls a shared Java evaluator. The Java compiler, `javac`, compiles that class and its supporting runtime. A separately written Python evaluator supplies another execution of the same proposed rule. The Catala translators take the same shared model when their capability report permits it: the compatibility route lowers supported RuleIR, while the native route emits richer Catala directly. Each target receives the same declared facts and assessment times for the cases in its profile.



The Java/Python pair and the Catala routes check whether a proposed rule survives translation within a declared profile. They can agree on a rule that omits a legal exception.

The test cases must also be capable of exposing mistakes. Boundary cases place values on either side of a threshold. Missing-fact cases check that uncertainty survives. Event histories exercise consent, withdrawal and timing. Deliberately altered rules, called *mutations*, ask whether the tests notice a removed exception or a reversed condition.

The successor also registers a narrow Lean certificate for Boolean expression lowering. Its fixed checker proves the relation between two formal descriptions; an additional finite check executes the generated Java for every declared Boolean state assignment within its size limit. Arithmetic, dates, full event histories and English meaning remain outside that theorem. A separate Java entry point demonstrates attributed contact histories, keeping the duty trigger distinct from an observed contact and from overall compliance.



A useful test notices a meaningful error. Mutation testing checks that ability; it does not manufacture independent legal answers.

For a small finite rule, every declared input combination can be enumerated. Larger

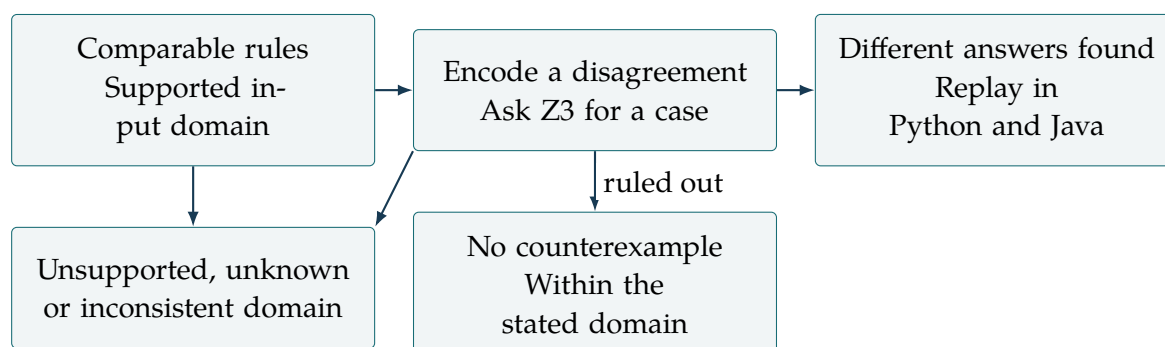
or unsupported domains require more limited conclusions. Expected legal outcomes need their own source justification, separate from Python/Java agreement. Compilation failures and disagreements return to the affected rule or implementation; successful checks accompany the draft rather than remove its open interpretation questions. Chapter 7 develops these checks and the separate Java host demonstration.

6 continued. Where mathematical verification fits

A mathematical check needs a precise proposition: whether two encoded rules differ on a permitted input, for example, or whether a transfer preserves a quantity. Such propositions concern the formal model and its assumptions.

Symbolic comparison with Z3

Z3 is a satisfiability modulo theories (SMT) solver: it searches for values that make a collection of logical and arithmetic conditions true. Here the condition asks whether two supported rule encodings return different answers. The facts, output meanings and permitted input domain must first be made comparable.

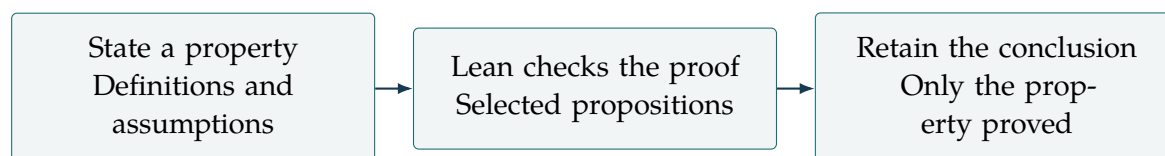


A solver result concerns an encoded comparison. An unsupported operation, timeout or empty domain cannot count as agreement.

The solver adapter covers a stated subset of the formal model and does not check an independent proof certificate from the solver. A found discrepancy is valuable because the original programs can replay it. A no-counterexample result retains the encoding and domain assumptions. Unsupported operations, timeouts and empty domains remain explicit outcomes. Chapter 3 explains those restrictions.

Proofs of selected mathematical properties

Lean is a proof assistant that checks a formal derivation. The mathematical review used it for selected logical and arithmetic propositions, including three-valued logic, an exception counterexample and conservation under a stated transfer. SymPy, a symbolic algebra system, checked selected algebraic identities.

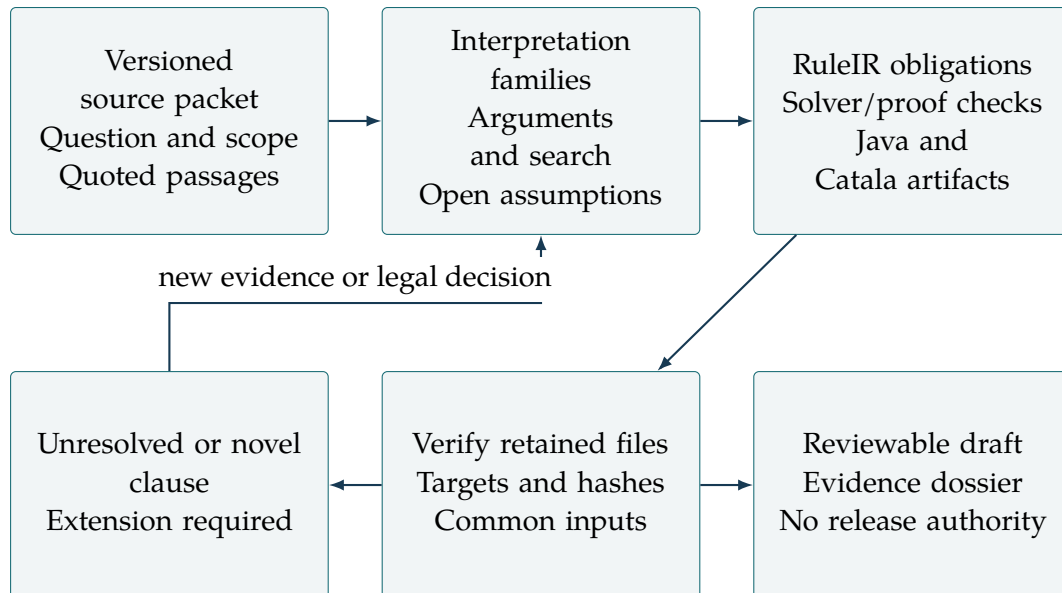


These are checks of selected mathematical claims, not an automatic proof of each generated program or of the whole compiler.

Proving that every supported rule preserves its behaviour through translation remains a stronger goal. The present checks do not establish that whole-compiler result or correct interpretation of English. The Stipula/JML/KeY route offers another research approach to particular Java properties.

The proof-carrying dossier and the abstention path

The preceding checks are useful only when their targets and assumptions stay visible. The integration record therefore binds a source packet, competing hypotheses, arguments, proof obligations, method-family results and executable artifacts by hash. It also records shared dependencies: agreement between Java and Catala after the same RuleIR lowering is a translation check, not two independent readings. The controller never changes an unavailable, unknown or unsupported result into agreement.



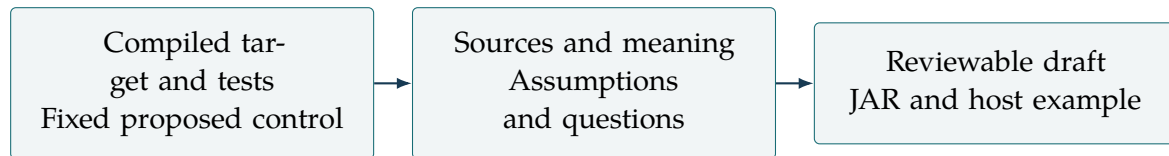
A proof or replay closes the proposition it names. An open authority, future clause, stale result or unavailable required method keeps the dossier on the abstention path.

The worked 23EC46 replay imports five previously proposed readings. Two differ on whether product-directed presentation suffices without an operational connection. The solver finds a separating assignment and the original Python and Java programs replay it. The Catala routes supply another execution target over the shared model: the compatibility backend consumes supported RuleIR, while the native converter can consume a richer model without first making it RuleIR. Agreement after a common lowering is a translation check, not independent legal evidence. The disabled live provider supplies no fresh reading. The witness identifies a legal question; it does not answer it. The result can be delivered as a draft Java package with its evidence and open questions, but the host must route the unresolved classification according to a bank-approved policy. The exact contract and master command are described in Section 6.34.

The master validates a frozen revision and binds each accepted phase to its inputs and predecessor files. Failure preserves the attempt and triggers a focused repair check before a bounded retry. Success refreshes the next-phase plan. A changed source, checker, plan or allowlist invalidates acceptance; retry exhaustion produces a report of the remaining failure. Neither a retry nor a successful document build supplies missing interpretive evidence.

7. Deliver a reviewable executable control

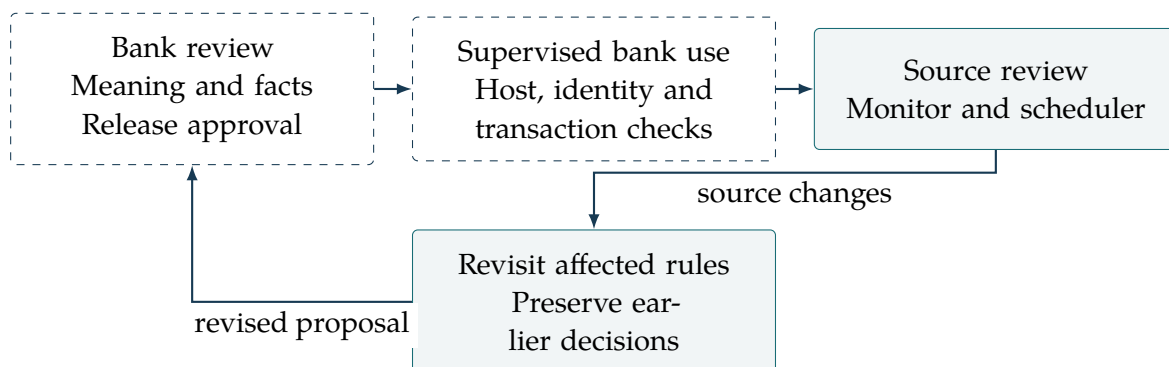
The result is a target program that another person can inspect, run and question. A Java archive (JAR) or a declared Catala package contains the compiled code. The associated review record identifies the sources, proposed meanings, facts, test results and unresolved questions. A separate host program exercises the callable control and preserves the target identity.



The delivered evidence lets a reviewer relate an executable answer to its proposed meaning. Draft delivery supplies no authority to use it.

8. Approve and supervise bank use

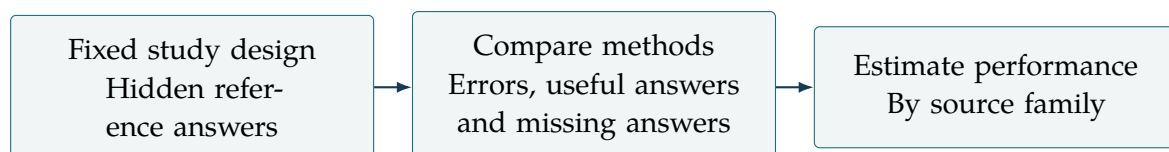
The institution decides which interpretation and exact program version it will authorize. Its own identities, fact owners and transaction systems must preserve that decision. The local source monitor and bounded scheduler can revisit affected controls; actual bank operation still needs institution validation.



Dashed boxes require the bank's operating evidence. Local monitoring is implemented; an unchanged source still leaves unresolved questions open.

Measuring whether the method helps

Empirical evaluation runs alongside development. The implemented comparison offers a single reader, isolated readers, tree search and source assurance under declared resource limits. Independent reference outcomes and unfamiliar source families are needed to estimate interpretation quality; a separate workflow study measures people's review effort.

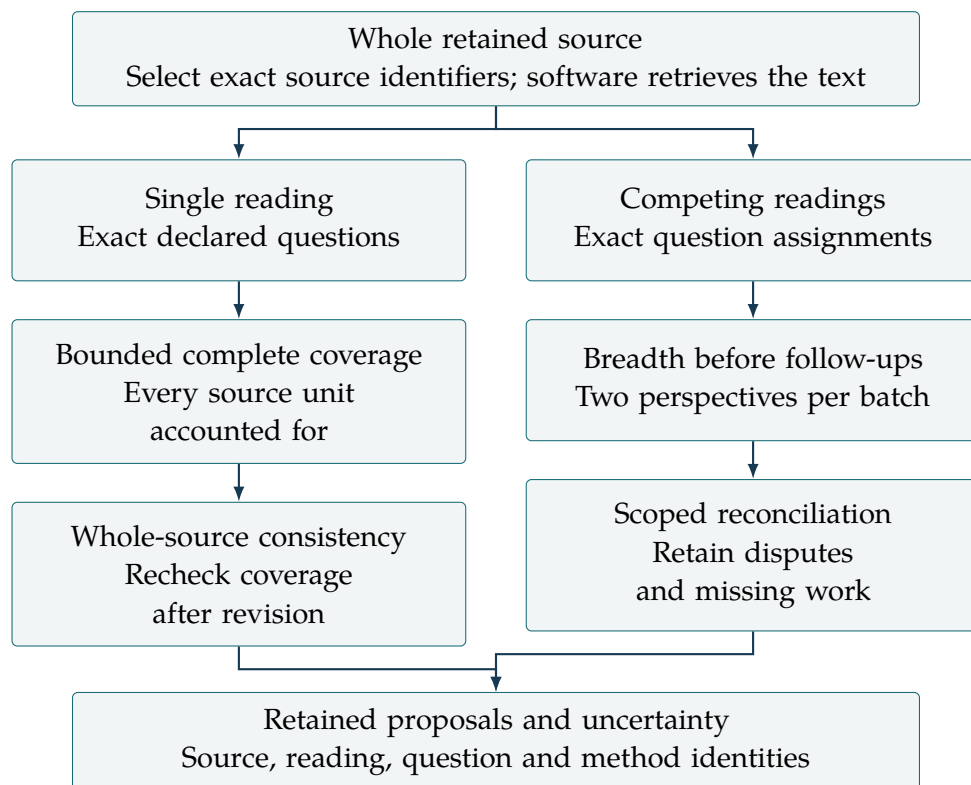


The comparison machinery exists. Its development checks have not established a method ranking or a reduction in reviewer workload.

Complete the processing without losing the question

The human-reference design above records an earlier research proposal. The current product requires a machine-checked proposition or an explicit qualification; human labels and approval do not establish product quality.

Long responses and exhausted review budgets can leave parts of a circular unexamined. Two implemented paths share exact source references. The single reader accounts for its declared questions; the ensemble investigates competing readings. These paths retain their separate results. See Section 6.36 for the mechanics and checked limits.



An interpretation revision reopens its coverage checks. Failed requests keep their spent attempts; no batch disappears to fit the budget.

The qualification adapter checks supported formal propositions for a retained reading and exact question. The complete ensemble investigation invokes it when enabled; the split single-reader path retains proposals for subsequent checking. A source identifier proves where the words came from. Coverage proves that the declared units and questions received dispositions. A formal theorem proves its stated relation. None of these alone proves the English interpretation. The four earlier unfamiliar investigations therefore remain incomplete unless their remaining stages actually execute under the recorded limits.